CAPSTONE PROJECT DOCUMENT

StreamMini

Product Requirements Document & System Architecture — Mini Streaming Platform (Frontend-First MVP)

Version: 1.0 — Draft for Team Review

Prepared for: Capstone Project Development Team

Date: September 2026

Focus: Tier-1 Frontend Application with Full Core Feature Set

Table of Contents

1. Introduction & Purpose — Part A — PRD
2. Problem Statement & Goals
3. Target Users & Personas
4. Scope (In / Out)
5. Functional Requirements
6. User Stories
7. Non-Functional Requirements
8. Success Metrics & Acceptance Criteria
9. Timeline & Milestones
10. Risks & Assumptions
11. System Architecture Overview — Part B — Architecture
12. Frontend Architecture (Deep Dive)
13. Component Breakdown
14. State Management & Data Flow
15. API Contract Overview
16. Tech Stack Summary
17. Folder Structure
18. Deployment Architecture

Part A — Product Requirements Document

1. Introduction & Purpose

StreamMini is a mini video streaming web application being built as a capstone project. The purpose of this document is to define **what** the product needs to do (the PRD) and **how** it will be built (the system architecture), so the development team shares one source of truth before implementation begins.

The primary deliverable for this phase of the project is a **Tier-1 frontend application**: a production-quality client that implements the complete core feature set of a streaming platform (browsing, playback, accounts, personalization, search) in a scaled-down but fully functional form. The backend is designed alongside it so the frontend has a real contract to build against, but the grading and demo emphasis is on the frontend experience.

2. Problem Statement & Goals

> Streaming platforms like Netflix or YouTube are large, complex systems. As a capstone project, the goal is to demonstrate — at a smaller, achievable scale — that the team can design and build a real, end-to-end streaming product: content catalog, authentication, playback, personalization, and a clean UI, all wired to a working API.

Academic Goals

- Demonstrate full-stack / frontend architecture skills
- Apply component-based UI design at production quality
- Show understanding of state management, routing, and API integration
- Produce documentation suitable for professional handover

Product Goals

- Let a user browse and discover video content easily
- Provide smooth, reliable video playback
- Support accounts, watchlists, and progress tracking
- Feel visually and functionally close to a real streaming app

3. Target Users & Personas

Persona	Description	Key Needs
Casual Viewer — "Ada"	Watches short-form and long-form videos in her spare time, browsing by mood.	Fast browsing, good recommendations, simple search.
Returning Binger — "Femi"	Watches series episode by episode over several days.	Continue-watching, progress saved, next-episode autoplay.
Content Curator / Admin — "Grader/Instructor"	Needs to inspect the catalog and upload/manage demo content.	Simple admin/upload flow, content management.

4. Scope

✓ In Scope (MVP)

- Email/password auth (sign up, login, logout)
- Home feed with categorized rows (trending, genres)
- Title detail page (synopsis, cast, episodes)
- Video player with core controls
- Search with filters (genre, type)
- Watchlist / "My List"
- Continue watching (resume position)
- Likes and basic comments
- Responsive design (desktop + mobile web)
- Minimal admin panel to upload/manage titles

⊘ Out of Scope (for this phase)

- Real adaptive bitrate/DRM-grade streaming infrastructure
- Native mobile apps (iOS/Android)
- Payment/subscription billing
- Multi-profile household accounts
- Offline downloads
- Recommendation ML models (use rule-based sorting instead)
- Live streaming

Items marked "out of scope" are natural stretch goals if time remains after the MVP is stable.

5. Functional Requirements

Priority key: MUST core to the demo SHOULD strongly expected COULD nice-to-have if time allows.

5.1 Authentication & Account

ID	Requirement	Priority
FR-1.1	User can register with name, email, and password.	MUST
FR-1.2	User can log in and log out; session persists via token.	MUST
FR-1.3	User can view/edit basic profile info and avatar.	SHOULD
FR-1.4	Password reset via email link.	COULD

5.2 Content Catalog & Discovery

ID	Requirement	Priority
FR-2.1	Home page shows curated horizontal rows (e.g. Trending, Continue Watching, By Genre).	MUST
FR-2.2	Title detail page shows poster/banner, synopsis, genre tags, cast, episode list (for series).	MUST
FR-2.3	Search bar with live/type-ahead results, filterable by genre and content type.	MUST
FR-2.4	"Similar titles" section on the detail page (simple tag-based matching).	COULD

5.3 Playback

ID	Requirement	Priority
FR-3.1	HTML5 video player with play/pause, seek, volume, fullscreen.	MUST
FR-3.2	Player resumes from last watched position.	MUST
FR-3.3	Autoplay next episode for series with a skip-intro / cancel option.	SHOULD
FR-3.4	Selectable video quality (e.g. 480p/720p) where multiple renditions exist.	COULD

5.4 Personalization

ID	Requirement	Priority
FR-4.1	User can add/remove titles from "My List".	MUST
FR-4.2	"Continue Watching" row driven by real watch-progress data.	MUST
FR-4.3	User can like a title and leave a short comment.	SHOULD

5.5 Admin / Content Management

ID	Requirement	Priority
FR-5.1	Admin can add a title (metadata + video file/URL + poster).	MUST
FR-5.2	Admin can edit or remove existing titles.	SHOULD
FR-5.3	Admin dashboard shows basic stats (views, top titles).	COULD

6. User Stories (Sample)

- As a **viewer**, I want to see trending titles on the home page so I can quickly find something to watch.
- As a **viewer**, I want the player to remember where I left off so I don't have to re-find my spot.
- As a **viewer**, I want to search by genre so I can narrow down choices when browsing feels overwhelming.
- As a **returning user**, I want a "Continue Watching" row so my in-progress shows are always one click away.
- As an **admin**, I want to upload a new title with a poster and video so the catalog can grow.

7. Non-Functional Requirements

Category	Requirement
Performance	Home page interactive in under 2.5s on a typical broadband connection; video should start playing within 2s of clicking play.
Responsiveness	Fully usable on desktop (≥1280px) and mobile web (≥360px) viewports.
Accessibility	Keyboard-navigable player controls, sufficient color contrast, alt text on images.
Security	Passwords hashed (bcrypt), auth via short-lived JWT + refresh token, input validation on all forms.
Browser Support	Latest two versions of Chrome, Firefox, Edge, Safari.
Scalability (design intent)	Architecture should not block scaling to real object storage / CDN later, even though the MVP uses simplified storage.
Maintainability	Component-based frontend, typed where possible, documented API contract.

8. Success Metrics & Acceptance Criteria

- All MUST functional requirements implemented and demonstrable end-to-end.
- A first-time user can go from landing page → sign up → browse → play a video in under 60 seconds without guidance.
- No critical (playback-blocking or auth-blocking) bugs at demo time.
- Lighthouse performance score ≥ 80 on the home page.
- Positive walkthrough from at least 2 peer testers before final submission.

9. Timeline & Milestones (Suggested 8-Week Plan)

Week	Milestone
1	Finalize PRD & architecture (this document), set up repos, design system, wireframes.
2	Auth flow (frontend + backend) and basic app shell/navigation/routing.
3	Home feed + title detail page wired to mock/real API.
4	Video player integration, resume/progress tracking.
5	Search, filters, My List, continue watching.
6	Likes/comments, admin upload panel.
7	Polish, responsive fixes, accessibility pass, bug bash.
8	Final QA, deployment, demo rehearsal, documentation handover.

10. Risks & Assumptions

Risks

- Video hosting/storage costs or limits for demo content.
- Player performance on lower-end devices/browsers.
- Scope creep beyond the 8-week window.

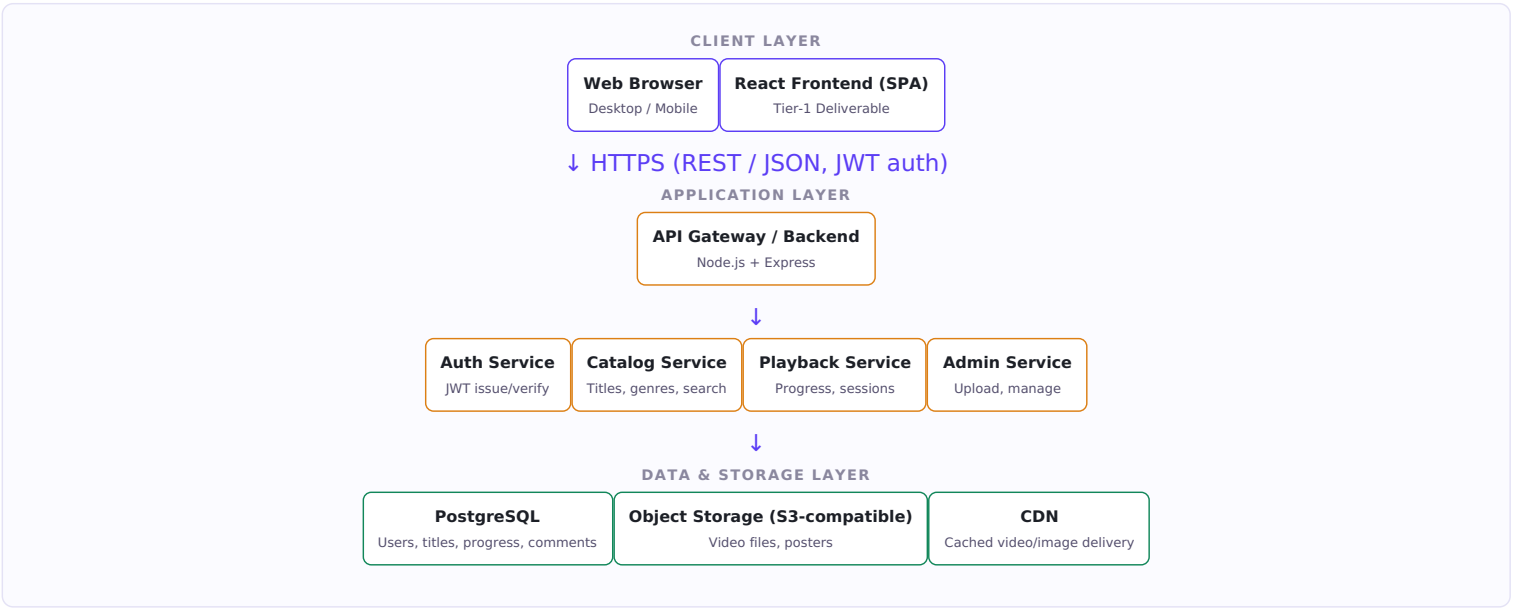
Assumptions

- Demo content will use royalty-free or team-produced videos.
- Team has basic familiarity with React and REST APIs.
- A small managed database/storage tier is available (free tier acceptable).

Part B — System Architecture

11. System Architecture Overview

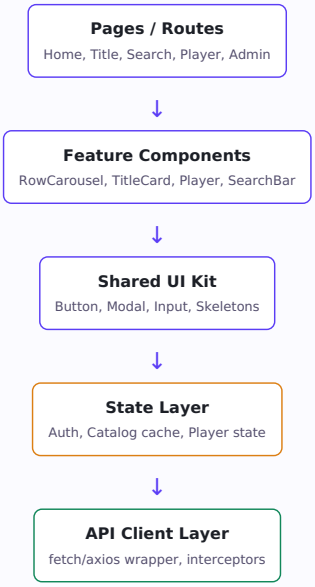
The system follows a standard client-server model: a React-based frontend (the primary deliverable), a REST API backend, a relational database for structured data, and object storage for video/image assets served through a CDN layer.



The frontend never talks to the database or storage directly — all data flows through the backend's REST API. Video files are uploaded via the Admin Service to object storage, then served to the client through CDN-cached URLs for fast playback.

12. Frontend Architecture (Deep Dive)

Since the frontend is the primary deliverable, it is architected as a layered single-page application:



12.1 Key Frontend Modules

Module	Responsibility
Routing	Client-side routes for Home, Title Detail, Search, Player, Auth, Admin (React Router).
Auth Context	Holds current user + token, exposes login/logout/register actions, guards protected routes.
Catalog Store	Fetches and caches title lists, genres, and search results.
Player Module	Wraps the HTML5 video element (or a lightweight library) with custom controls and progress reporting.
API Client	Centralized wrapper around HTTP calls with base URL, auth header injection, and error handling.
UI Kit	Reusable presentational components used across every page for visual consistency.

13. Component Breakdown (Selected Screens)

Home Page

- NavBar (search, profile menu)
- HeroBanner (featured title)
- RowCarousel × N (Trending, Continue Watching, By Genre)
- TitleCard (poster, hover preview, quick-add to list)

Title Detail Page

- TitleHeader (banner, title, meta, play/list buttons)
- EpisodeList (for series)
- CommentsSection
- SimilarTitlesRow

Player Page

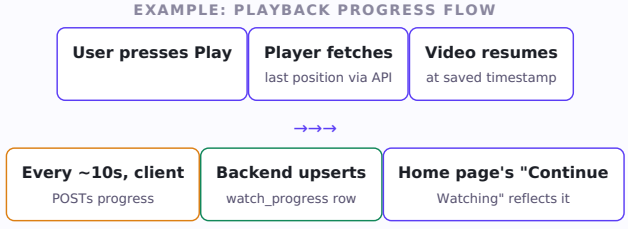
- VideoPlayer (controls, progress bar, quality menu)
- NextEpisodeOverlay
- ProgressReporter (background sync to API)

Admin Panel

- TitleForm (create/edit metadata)
- MediaUploader (video + poster upload)
- TitleTable (list, edit, delete)

14. State Management & Data Flow

Recommended approach: React Context for global concerns (auth session, theme), combined with a lightweight data-fetching library for server state.



Concern	Recommended Tool
Auth/session state	React Context + localStorage-persisted token
Server data caching (catalog, search)	React Query / TanStack Query (or SWR)
Local UI state (modals, form inputs)	Component-local useState
Cross-cutting player state	Small dedicated Player Context or Zustand store

15. API Contract Overview (Key Endpoints)

Method & Path	Purpose
`POST /api/auth/register`	Create a new user account.
`POST /api/auth/login`	Authenticate, return access + refresh tokens.
`GET /api/titles`	List/browse titles, supports genre/type filters.
`GET /api/titles/:id`	Get full detail for one title (episodes, cast, etc.).
`GET /api/search?q=`	Search titles by keyword.
`GET /api/me/list` / `POST` / `DELETE`	View, add to, or remove from "My List".
`GET /api/me/progress` / `POST`	Fetch or update watch progress for a title.
`POST /api/titles/:id/comments`	Add a comment to a title.
`POST /api/admin/titles` / `PUT` / `DELETE`	Admin create/update/delete a title.

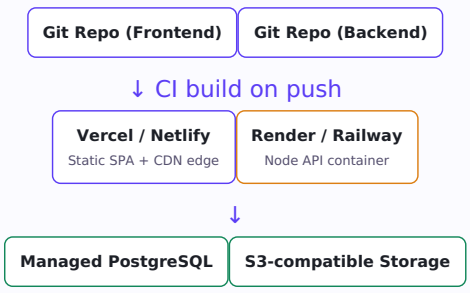
16. Tech Stack Summary

Layer	Choice	Why
Frontend Framework	React + TypeScript	Component model fits a card/row-based UI; TS catches contract mismatches early.
Styling	Tailwind CSS	Fast, consistent styling for a media-heavy grid UI.
Routing	React Router	Standard SPA routing.
Server-state caching	TanStack Query	Handles caching, loading/error states for API calls out of the box.
Video Playback	HTML5 `<video>` (+ hls.js if adaptive streaming is attempted)	No licensing overhead; hls.js adds adaptive bitrate later without a rewrite.
Backend	Node.js + Express	Same language as frontend, quick to stand up a REST API.
Database	PostgreSQL	Relational data (users, titles, episodes, progress) fits well; strong tooling.
File/Video Storage	S3-compatible bucket (e.g. AWS S3, Cloudflare R2, or Supabase Storage)	Cheap, simple, swappable later for a real CDN pipeline.
Auth	JWT (access + refresh)	Stateless, simple to reason about for a small team.
Hosting (Frontend)	Vercel / Netlify	Zero-config deploys, previews per PR.
Hosting (Backend)	Render / Railway	Free/low-cost tiers suitable for a capstone budget.

17. Folder Structure (Frontend)

```
src/
  assets/                    // static images, icons
  components/               // shared UI kit (Button, Modal, Input...)
  features/
    auth/                // LoginForm, RegisterForm, AuthContext
    catalog/            // RowCarousel, TitleCard, TitleDetail
    player/             // VideoPlayer, ProgressReporter
    search/             // SearchBar, SearchResults
    admin/              // TitleForm, MediaUploader, TitleTable
  pages/                   // HomePage, TitlePage, PlayerPage, AdminPage
  lib/
    apiClient.ts        // axios/fetch wrapper + interceptors
  hooks/                  // useAuth, useTitles, useProgress
  router.tsx
  App.tsx
```

18. Deployment Architecture



This document is a living draft — intended to be reviewed with the full team, adjusted, and versioned as the project progresses through each milestone.